

Chapter 5 Software Design Architecture

- A software system is **made of multiple structures** to show how the system is built and functions 由多种结构构成, 以展示系统的构建和功能
- Each structure **includes components** and the **relationships** between them.
- Software architecture is the **set of structures used to understand the system**.
- It includes software parts, their visible features, and how they interact. 包含软件组件、其可见特性以及它们之间的交互方式
- **Helps us understand how parts are organized, interact, and affect system performance.** 帮助我们理解组件的组织方式、交互方式以及它们对系统性能的影响
- Interaction methods: Pipes, Client-Server, Peer-to-Peer, etc.
- System-wide properties: Data flow speed, delay (latency), how changes spread across the system.

Three Categories of Structures

1. Module Structures (Static View) 模块结构(静态视图)

- Divides systems into **modules** for **easier design, coding, and team collaboration**.
- Each module **handles specific logic or function** (e.g., UI, business logic, database).
- Common in **large systems** where work is split among teams. 工作由多个团队分工完成
- **Class diagrams** show module structures in object-oriented design.
- Modules are static; they show how work is split among teams. 显示团队之间工作是如何分工
- Types of module structures:
 - **Decomposition** 分解: A module includes smaller submodules.
 - **Uses** 使用: A module uses another for support or resources. 作为支持或资源
 - **Layered** 分层: Modules are arranged in levels/layers. 按级别/层级排列
 - **Class-based**: Uses object-oriented concepts (inheritance, instances).

Example: In ERP software, database logic, UI, and business rules are handled by different teams/modules.

2. Component-and-Connector Structures (Dynamic View)

- Focuses on **how software parts communicate at runtime**. 关注软件组件在运行时如何通信
- Emphasizes interaction, data flow, and execution. 强调交互、数据流和执行
- “Component” refers to something that runs at runtime. 是指在运行时运行的组件
- Types of Component-and-Connector Structures
 - **Concurrency** 并发: Multiple parts run in parallel (multithreading). 并行运行
 - **Shared Data** 共享数据: Components read/write shared data. 排列
 - **Client-Server** : One part requests, another responds. 排列
 - **Process-based**: Components are connected by message passing, sync, or exclusion.

Example: Video streaming app — playback component interacts with network streaming and buffering. 播放组件与网络流和缓冲交互

3. Allocation Structures 分配结构

- Shows **how software elements are assigned to real-world resources**. 软件元素如何分配给实际资源
- Helps with deployment, team planning, and organizing codebases.
- Examples of Allocation Structures
 - **Deployment**: Maps software to physical machines/hardware. 将软件映射到物理机器/硬件
 - **Implementation**: Shows file structure or how code is stored.
 - **Work Assignment**: Assigns tasks or modules to development teams. 分配任务/模块

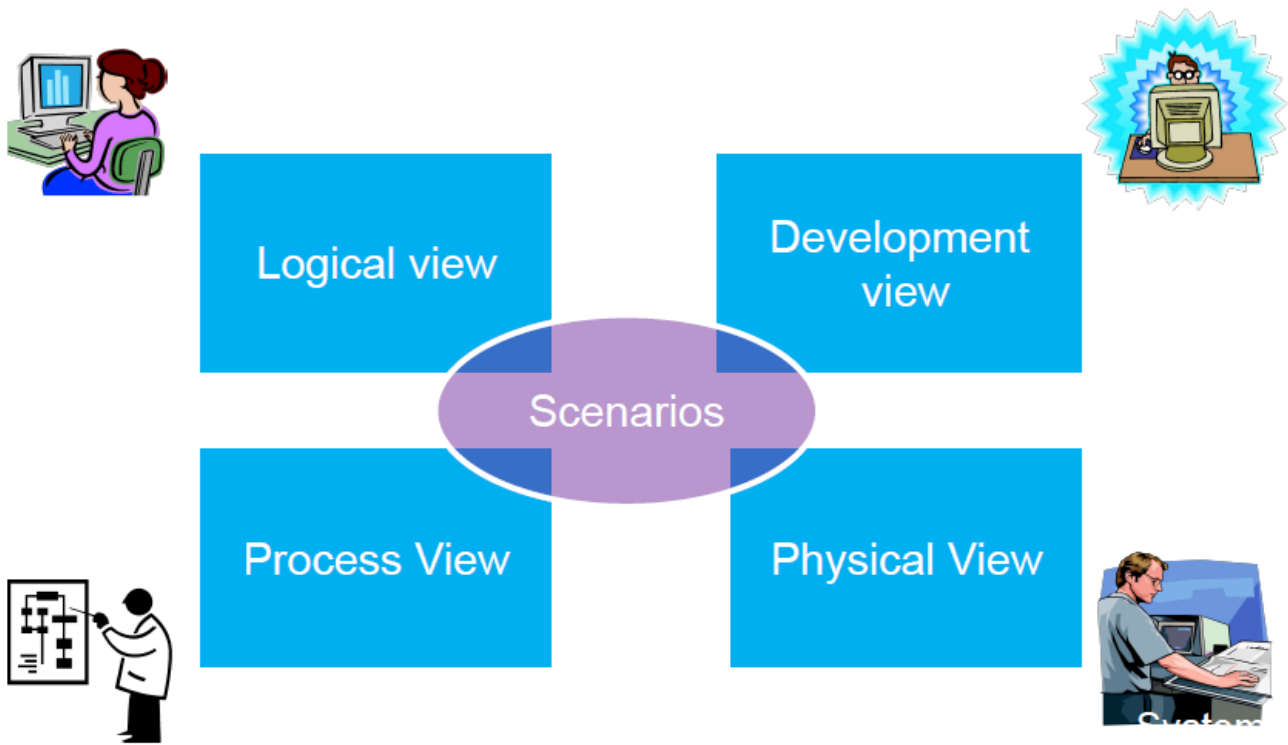
Example: Backend module runs on a cloud server, UI module built by frontend team.

Relating Structures 关联结构

- Each structure gives a **unique perspective** of the system. 提供了系统的独特视角
- All structures are **connected** and **influence** each other. 相互关联& 影响
- **Not all systems use every structure** — large systems usually require more. 大型系统通常需要更多结构
- Choosing and combining structures helps **satisfy quality goals** (e.g., performance, maintainability).
- Helps apply Separation of Concerns — different teams can focus on different parts. 有助于应用关注点分离——不同的团队可以专注于不同的部分

Architectural View

- Modern software systems are **complex** and cannot be understood from one view alone. 无法仅从单一视角理解
- Architectural views help **simplify** and **describe** the system from **different perspectives**. 有助于从不同角度简化和描述系统
- Each view focuses on different concerns for different stakeholders (users, developers, testers, etc.). 关注不同利益相关者的不同关注点
- **4+1 View Model** (by Philippe Kruchten) introduces 4 core views + 1 use case view to link them together.



Kruchten's 4+1 Model Views

Logical View – Object-Oriented Decomposition - Shows key concepts (modules).

- Also called Design or Conceptual View.
- Describes the **main components** (classes, objects, modules) and their relationships.
描述主要组件及其关系
- Shows **system functionality** from the user or developer perspective.
从用户或开发者的角度展示系统功能
- Based on **object-oriented principles**: abstraction, encapsulation, inheritance.
- Used during early system design and documentation.
用于早期系统设计和文档编制
- Commonly shown using **class diagrams** and **package diagrams**.

Use when: Understanding system functions, designing based on requirements. 理解系统功能, 基于需求进行设计

Viewer: End-user

Focus: What the system should provide to users

- ✓ Easy to map requirements to modules. 易于将需求映射到模块
- ✓ Helps developers understand and reuse core components. 帮助开发者理解和重用核心组件
- ✗ Doesn't show how components run at runtime. 不展示组件在运行时的运行方式
- ✗ May become complex for large systems.

Process View – Process Decomposition - Shows concurrency and distribution.

- Focuses on **runtime behavior**, especially concurrency and communication between processes. 关注运行时行为, 尤其是进程间的并发性和通信
- Describes **how components work together dynamically** 如何动态协同工作 (e.g., threads, tasks, sync).
- Addresses **non-functional** needs 满足非功能性需求 like performance, availability, fault-tolerance. 容错
- Shown using **activity diagrams, communication diagrams**, or sequence flows.

Use when: Designing systems with parallel processes (e.g., real-time systems, servers). 设计具有并行进程的系统

Viewer: Integrators

Focus: Non-functional needs like performance and concurrency

- Representation:
 - Multiple abstraction levels 多个抽象级别
 - Processes and threads
 - Major and minor tasks
 - Communication methods like messaging, RPC, events
 - Minor tasks use shared memory or rendezvous 次要任务使用共享内存或集合点
- ✓ Reveals performance bottlenecks and runtime issues. 揭示性能瓶颈和运行时问题
- ✓ Supports analysis of multi-threaded or distributed behavior. 支持多线程或分布式行为的分析
- ✗ Harder to visualize for beginners 可视化难度较大
- ✗ May not reflect how system is organized in code. 无法反映系统在代码中的组织方式

Development View – Subsystem Decomposition- Shows module layout for building

- Also called Implementation View
- Describes **how the system is structured in code** (folders, modules, packages). 描述系统在代码中的结构
- Useful for software **building, testing, and maintenance**.
- Shows the **layered architecture**, dependencies, and component breakdown. 显示分层架构、依赖关系和组件分解
- Used by **programmers and software managers**

Use when: Organizing codebase, assigning coding tasks, managing builds.

Viewer: Programmers and software managers

Focus: Software module layout, tool use, reuse

- Representation:
 - Layered style (depends on lower layers)
- ✓ Supports team collaboration and version control. 支持团队协作和版本控制
- ✓ Helps identify reusable components. 帮助识别可复用组件
- ✗ Doesn't show runtime behavior. 不显示运行时行为
- ✗ May not align clearly with user features. 可能与用户功能不一致

显示部件的运行位置

Physical View – Mapping Software to Hardware - Shows where parts run.

- Also called **Deployment View**
- Maps software components to **hardware infrastructure** (servers, devices, networks).
将软件组件映射到硬件基础设施
- Focuses on **scalability, reliability, fault tolerance**. 可扩展性、可靠性和容错性
- Includes details like **network topology, nodes, processes, and physical connections**.

Use when: Planning system deployment, infrastructure setup, and system reliability.

Viewer: System engineers

Focus: Non-functional needs like availability and hardware communication

- Representation:
 - Diagrams based on the process view 基于进程视图的图表
 - May have test/development and deployment architectures 可能包含测试/开发和部署架构
- ✓ Helps optimize system performance and resource usage. 有助于优化系统性能和资源利用率
- ✓ Important for large systems with many servers or locations.
- ✗ Tied to specific hardware; changes in infrastructure may affect design. 与特定硬件绑定; 基础设施的变化可能会影响设计
- ✗ Adds extra complexity during early development. 增加额外的复杂性

Scenarios / Use Case View (+1)- use cases to make sure the views work together.

- Combines and validates the 4 views using real-life **use cases or scenarios**.
- Shows **how the system behaves** in actual situations. 显示系统在实际情况下的行为方式
- Helps **verify that the architecture meets requirements** and that views don't conflict.
帮助验证架构是否满足需求, 且视图之间不存在冲突

Use when: Testing the system design, discussing with stakeholders, or validating architecture.

Viewer: Everyone and evaluators

Focus: Supports other views

- Purpose:
 - Help find and validate architecture parts 帮助查找和验证架构部分
 - Check if the system is correct
- Representation:
 - Similar to logical view with some differences
- ✓ Ensures consistency and correctness of architecture. 确保一致性和正确性
- ✓ Helps in requirement tracing and early design testing. 有助于需求追踪和早期设计测试
- ✗ Needs good use case writing to be useful.
- ✗ May not fully cover all edge cases or complex logic

Correspondence Between the Views

- The 4+1 architectural views are **connected and not isolated** — each view supports and complements the others. 相互关联而非孤立——每个视图都相互支持和补充
- Designing a system involves moving between these views to ensure consistency and complete understanding. 系统设计需要在这些视图之间切换, 以确保一致性和完整的理解

- You typically start from the **Logical View** (user requirements), then build the **Development** or **Process View**, and end with the **Physical View** (deployment setup).
- a. From Logical to Process:
 - Used to design the **runtime behavior** based on the system's static structure.
用于基于系统静态结构设计运行时行为
 - Helps identify parts that need to run in parallel or manage time-sensitive actions.
帮助识别需要并行运行或管理时间敏感操作的部分
 - ✓ Gives control over runtime behavior early in design. 控制运行时行为
 - ✓ Improves performance and resource handling in complex systems. 提升复杂系统的性能和资源处理能力
 - ✗ Hard to manage if the logical view is too abstract. 过于抽象, 则难以管理
 - ✗ Needs a deep understanding of system execution. 需要深入了解系统执行过程
 - Two ways to study concurrency:
 - **Inside-Out**: Start from Logical View → decide what needs to run concurrently.
确定需要并发运行的内容
 - **Outside-In**: Start from deployment (Physical View) → then plan how logic runs in parallel.
然后规划逻辑如何并行运行
- b. From Logical to Development
 - Helps translate system functionality into **code structure and file layout**.
帮助将系统功能转化为代码结构和文件布局
 - These two views are closely linked but can differ in large or evolving systems.
这两个视图紧密相连, 但在大型或不断发展的系统中可能会有所不同
 - ✓ Maintains consistency between design and implementation. 保持一致性
 - ✓ Helps in assigning coding tasks to teams.
 - ✗ In large projects, the gap between design and code structure may grow. 差距可能会越来越大
 - ✗ Difficult to maintain alignment if requirements change often. 如果需求经常变化, 则很难保持一致性
 - Grouping into subsystems can be based on:
 - Classes
 - By related **classes or class packages**
 - By **lines of code (LOC)**
 - Based on **team responsibilities**

Architectural Styles

- An **architectural style** is a general way of organizing a system's components and how they connect. 是组织系统组件及其连接方式的通用方法
- It gives a **standard structure** that developers can follow to solve common design problems. 提供了一种可以遵循的标准结构来解决常见的设计问题
- It's not actual code, but a **set of rules and patterns** to structure systems. 并非实际的代码, 而是一套用于构建系统的规则和模式
- Styles guide **how components interact, how control flows, and how data moves.** 风格指导组件如何交互、控制流如何流动以及数据如何传输
- Using a proper style improves **system quality, development speed, and team coordination** 使用合适的风格可以提高系统质量、开发速度和团队协作能力
- Architects can choose different styles for different views.

The choice depends on:

- **System type** (web app, desktop app, distributed system, etc.)
- **Requirements** (performance, security, maintainability)
- **Desired quality attributes** (scalability, flexibility, etc.)

What Is an Architectural Style

- A style describes a **family of system designs** that share the same organization pattern. 风格描述了一系列具有相同组织模式的系统设计
- Used repeatedly in real-world projects. 反复使用
- Provides a **flexible "template"** or structure for solving software problems.
- ✓ **Makes design faster by reusing proven solutions.** 加快设计速度
- ✓ **Easier for teams to understand and collaborate.** 更易于团队理解和协作
- ✗ Too generic — may need adaptation to specific systems. 过于通用—需要根据系统进行调整
- ✗ Choosing the wrong style can lead to inefficiency or complexity. 选择错误的风格会导致效率低下或复杂性增加

Styles样式 vs. Patterns模式

- Styles are broad and general 广泛而通用— they define structure and relationships. 定义结构和关系
- Patterns are more specific 则更具体— they give a step-by-step solution for a particular problem using the style. 为特定问题提供分步解决方案

Think of a style as the skeleton 骨架, and a pattern as the muscles and joints that bring it to life.

Use of Styles

- Developers must understand core styles to make informed decisions and avoid poor designs. 必须了解核心样式才能做出明智的决策并避免糟糕的设计
- Many styles are **converted** into patterns and reused in modern software architecture.
- ✓ **Helps understand pros/cons before applying the style.**
- ✓ **Increases reusability and system clarity.** 提高可复用性和系统清晰度
- ✗ Requires proper training and experience to choose the best one. 需要适当的培训和经验才能选择最佳样式
- ✗ Mixing styles without a plan can lead to inconsistent architecture. 在没有计划的情况下混合使用样式会导致架构不一致

What Is an Architectural Pattern 架构模式

- A **reusable solution** built on a style, with more concrete details. 基于某种风格构建的可复用解决方案, 但包含更具体的细节
- A pattern includes:
 - Component types (e.g., client, server, data store)
 - Layout/structure of components
 - Design rules and interaction mechanisms (e.g., message passing)
- Architectural patterns are like design patterns but cover more of the system 架构模式类似于设计模式, 但涵盖了更广泛的系统层面

Example: Pipe-and-Filter is a pattern based on the Data Flow style.

- ✓ Easy to apply and customize in real projects. 易于在实际项目中应用和定制
- ✓ Solves well-known problems efficiently. 高效解决常见问题
- ✗ May not fit unique or complex systems without change. 如果不进行修改, 可能无法适应独特或复杂的系统
- ✗ Can become rigid if overused without adjustment. 如果过度使用而未进行调整, 可能会变得僵化

Pattern Classification

1. Hierarchical Systems 分层系统

- Components are arranged in **layers or levels**, each with its own responsibility. 组件按层级或级别排列, 每个组件都有各自的职责
- Each level may include more subcomponents to perform specific tasks. 执行特定任务
- Follows **"top-down" design** — higher levels control or use lower levels.
- Two common patterns:
 - Main program and subroutine
 - Layered architecture

Use when: You want to organize logic cleanly and separate responsibilities 组织逻辑并分离职责

- ✓ Easy to manage and understand due to clear structure. 结构清晰, 易于管理和理解
- ✓ Encourages modularity and task separation 鼓励模块化和任务分离
- ✗ Top layer failure may affect the whole system. 顶层故障可能会影响整个系统
- ✗ Not suitable if components need frequent cross-layer interaction. 组件需要频繁的跨层交互

Main Program and Subroutine 主程序和子程序

- Top module is the **main controller**, calling other submodules as needed.
- Lower modules **share data** or perform specific operations for the main program. 执行主程序的特定操作

Use when: Using procedural programming or when one component controls the rest. 使用过程式编程或一个组件控制其他组件的情况

- **Modifiability** 可修改性: Easier to understand and manage due to independent parts.
- **Reusability** 可修改性: Independent parts can be reused elsewhere.
- ✓ Easy to understand and break into small tasks. 易于理解并分解成小任务
- ✓ Components are reusable and maintainable. 可重用且易于维护
- ✗ Main program becomes a bottleneck if overloaded. 如果主程序过载, 则会成为瓶颈
- ✗ Difficult to modify data flow without affecting many subroutines. 难以在不影响的情况下, 修改数据流

Layered Architecture 分层架构

- Groups related functions into **layers**, stacked vertically. 将相关功能分组到垂直堆叠的层中
- Each layer **uses the layer below** and **serves the layer above**. 每层都使用下一层, 并为上一层提供服务
- Example systems: Operating systems, network protocol stacks.

Use when: You need separation between UI, logic, and database or when multiple teams work on different layers.

- ✓ Layers are independent and easy to test/replace. 层之间相互独立, 易于测试/替换
- ✓ Supports parallel development and multi-level security. 支持并行开发和多级安全性
- ✗ Data may pass through many layers, causing slow performance. 性能下降
- ✗ Layers must be well-defined; too many = complexity, too few = mess. 层必须定义明确; 层数过多 = 复杂, 层数过少 = 混乱

When to Use Layered Architecture

- When building new features on top of existing systems 在现有系统之上构建新功能
- When different teams work on different layers 不同团队在不同的层级上工作
- When multi-level security is needed 需要多级安全性

Three-Layered Architecture

- Common in business systems. 常见于业务系统
- Has three parts:
 - Presentation: UI, display, and input.
 - Business: Rules, checks, and behaviors. 规则、检查和行为
 - Data Access: Talks to the database using SQL and APIs.

Use when: You want to separate display, processing, and data clearly. 清晰地分离

- ✓ Easy to change one layer without affecting others. 易于更改某一层而不影响其他层
- ✓ Supports team-based development. 支持团队开发
- ✗ More code may be needed to pass data through layers. 可能需要更多代码才能在各层之间传递数据
- ✗ Not ideal for very small or simple apps 不适用于非常小或简单的应用

Four-Layered Architecture

- Splits business logic further into:
 - Application Logic 应用逻辑(workflow, coordination)
 - Domain Logic 领域逻辑(core rules, calculations)
- Promotes **code sharing and reuse** across different applications.

Use when: You want clean logic separation and plan to reuse core rules in multiple apps.

- ✓ Increases modularity and code reuse.
- ✓ Clean separation improves testability and design clarity. 提高可测试性和设计清晰度
- ✗ Adds complexity and more layers to manage. 增加了复杂性和更多需要管理的层级
- ✗ Requires good planning and team coordination.

Advantages of Layered Architecture

- **Abstraction**
 - Allows understanding of the system layer by layer without needing to know internal workings of other layers. 允许逐层理解系统, 而无需了解其他层的内部工作原理
- **Encapsulation**
 - Hides internal details (like data types and logic) within each layer, improving security and clarity. 隐藏每层内部的细节 (例如数据类型和逻辑), 从而提高安全性和清晰度
- **Clear Responsibilities**
 - Each layer handles a specific set of tasks, making the system easier to maintain and understand. 每层处理一组特定的任务, 使系统更易于维护和理解
- **Controlled Communication**
 - Upper layers send requests to lower layers; lower layers send responses upward. 上层向下层发送请求; 下层向上发送响应
- **High Cohesion**
 - Each layer focuses on related functionality, which makes the internal code well-organized. 每层专注于相关功能, 使内部代码井井有条
- **Loose Coupling**
 - Layers connect through interfaces and abstractions, making them less dependent on each other. 各层通过接口和抽象连接, 从而减少彼此之间的依赖

Disadvantages of Layered Architecture

- **Reduced Performance**
 - Data must pass through multiple layers, which adds processing time. 数据必须经过多个层级, 这会增加处理时间
- **More Dependency in Open Layers**
 - If layers are allowed to skip others (open architecture), it increases complexity and tight coupling. 如果允许层级跳过其他层级 (开放式架构), 则会增加复杂性和紧耦合性
- **Debugging Difficulty**
 - Errors can be hard to trace because they pass through several layers. 由于错误会经过多个层级, 因此难以追踪
- **Too Few Layers**
 - Layers may become too large and unmanageable, reducing modularity. 层级可能变得过大且难以管理, 从而降低模块化程度
- **Too Many Layers**
 - Increases system complexity and can slow down data flow, affecting responsiveness. 会增加系统复杂性, 并可能减慢数据流速度, 影响响应速度

Partitioning 分区

- Complex layers can be broken down into **smaller sub-packages** to improve clarity and manageability. 以提高清晰度和可管理性
- Each sub-system within a layer should have a **clear interface** and **defined boundary**. 应具有清晰的接口和明确的边界
- Each part (subsystem or component) should focus on a **single job** or a **related set of services**. 应专注于一项任务或一组相关的服务
- Loose coupling between sub-parts helps with **better modularity and testability** 有助于提高模块化和可测试性

Quality Properties for Layered Architecture

- **Modifiability**: You can make changes in one layer without affecting others, allowing easier updates and maintenance. 可以在某一层级中进行更改而不会影响其他层级, 从而更轻松地进行更新和维护
- **Portability**: Services can interact directly with platform APIs, making the system adaptable to different environments. 服务可以直接与平台 API 交互, 使系统能够适应不同的环境
- **Security**: Each layer can apply its own encryption/decryption or access control measures. 每层级都可以应用自己的加密/解密或访问控制措施
- **Reusability**: Common services and logic in lower layers can be reused across multiple systems or projects. 较低层级中的通用服务和逻辑可以在多个系统或项目之间重用

2. Data-Centered Systems 数据中心系统

- Also called Shared Memory systems. 共享内存系统
- Focuses on a **central data store** that many components access and use. 专注于多个组件访问和使用的中央数据存储
- All communication between components happens **through shared data**, not direct interaction.
- Components can **add, read, update, or delete** data from the central store.

*Use when: Many subsystems or tools need to work with **the same data**, 需要处理相同的数据 such as in business systems, data analysis tools, or knowledge-based systems.*

Two Common Approaches

- **Repository** 存储库: All components use a **single shared central database**. 所有组件使用单个共享的中央数据库
- **Distributed Data** 分布式数据: Each component has its **own database** and sends data to others as needed 每个组件都有自己的数据库, 并根据需要将数据发送给其他组件

Blackboard Architecture

- Clients use a central repository. 客户端使用中央存储库
- Uses a central **blackboard** (shared memory/database) where different components work together to solve a problem. 使用中央黑板(共享内存/数据库), 不同组件在此协同工作以解决问题
- **Knowledge sources** 知识源(specialized modules) post or update data on the blackboard.
- A **control component** manages which knowledge source runs and when. 管理哪个知识源的运行及其运行时间
- Blackboard model has:
 - Blackboard: Shared memory with solution data 包含解决方案数据的共享内存
 - Knowledge sources: Specialized modules 专用模块
 - Control component: Chooses and runs modules 选择并运行模块
- In passive 被动 repositories, clients read/write data independently.
- In active blackboards, changes trigger updates for clients. 更改会触发客户端的更新
- Clients can be added or changed without affecting others.
- Clients can share data through the blackboard and work separately.

Use when: You need flexible problem-solving involving multiple expert modules (e.g., AI, expert systems). 需要灵活地解决涉及多个专家模块(例如, 人工智能、专家系统)的问题

Advantages of Blackboard

- **Efficient Data Sharing:** All components can access needed data from one place without duplication. 所有组件均可从一处访问所需数据, 无需重复
- **Loosely Coupled Clients:** Components don't need to know about each other — just the shared data format. 只需了解共享数据格式
- **Backup & Safety:** Central storage can have secure backup mechanisms. 可拥有安全的备份机制
- **Scalability & Reusability:** New clients/modules can be added easily without changing others.

Disadvantages of Blackboard

- **Shared Data Model Issue:** All components must agree on the same data structure, which can be hard to design. 所有组件必须统一使用相同的数据结构, 这在设计上可能比较困难
- **High Change Impact:** Changing the data format affects all components using it. 更改数据格式会影响所有使用该格式的组件
- **Poor Distribution:** Difficult to split across multiple machines or networks. 难以在多台机器或网络之间进行拆分
- **Costly Communication:** Transferring large data over a network is slow and resource-heavy 通过网络传输大量数据速度慢且耗费资源

Quality Properties of Blackboard

- **Modifiability**: Easy to add or remove components without touching the rest.
- **Reusability**: Specialized modules can be reused in other systems that share the same data format.
- **Maintainability**: Components are independent and easier to maintain separately.

3. Data Flow Systems

- Focuses on **how data moves and transforms** through a sequence of processing steps.
关注数据如何通过一系列处理步骤进行移动和转换
- Data flows from one component to another, where each component performs a specific task. 数据从一个组件流向另一个组件, 每个组件执行特定的任务
- The output of one step becomes the input of the next step.

*Use when: The system needs to process data in **stages**, like compiling code, video editing, or processing sensor data.* 系统需要分阶段处理数据

Pipe and Filter Architecture

- **Filters**: Independent components that process or transform data.
过滤器: 处理或转换数据的独立组件
- **Pipes**: Connections that carry data from one filter to the next.
管道: 将数据从一个过滤器传输到另一个过滤器的连接
- Each filter works independently and doesn't need to know about others.
每个过滤器独立工作, 无需了解其他过滤器的存在

Pipe and Filter – Batch Sequential 批量顺序

- If filters are arranged in a single line, it's called batch sequential.
- It takes a batch of data and applies filters one after another. 接收一批数据, 并逐个应用

Example: In an image editor:

- Filter 1: Resizes image
- Filter 2: Adds watermark
- Filter 3: Compresses file

Advantages of Pipe and Filter

- **Modifiability**: Easy to add, remove, or change filters without affecting others.
- **Reusability**: Filters can be reused in different systems or processing flows.
- **Parallelism**: Filters can run at the same time, speeding up the process.
并行性: 过滤器可以同时运行, 从而加快处理速度
- **Maintenance-Friendly**: Clear and isolated logic in each component.
逻辑清晰且相互隔离

Disadvantages of Pipe and Filter

- **No Direct Communication:** Filters can't interact with each other directly.
无法直接交互
- **Data Format Compatibility:** Each filter must accept and produce data in compatible formats.
以兼容的格式接收和生成数据
- **Difficult Error Handling:** Errors must be passed through pipes, making handling complex.
错误必须通过管道传递, 处理过程复杂
- **Limited for Real-Time Use:** Not ideal for time-critical or highly interactive systems.
不适用于时间要求严格或高度交互的系统

4. Distributed Systems 分布式系统

- The system is spread across **multiple computers or networked components** that communicate with each other. 系统分布在多台计算机或相互通信的联网组件上
- Components **work together** but may run on **different machines** or in different locations.
- Example styles: Client-server, N-tier, Broker, Peer-to-peer, SOA.

*Use when: The system needs to be **scalable**, work **over the internet**, or support **many users** at the same time. 系统需要可扩展、通过互联网运行或同时支持多用户*

Client-Server Architecture

- Also called 2-tier architecture.
- System has a client and a server.
 - **Client:** Sends requests and interacts with users.
 - **Server:** Processes data and provides services.
- Clients and servers run independently and communicate over a network (e.g., HTTP, RMI).

Example: A web browser (client) requests data from a website server.

- ✓ Easy to manage data and users centrally. 易于集中管理数据和用户
- ✓ Server upgrades can improve service without changing the client. 服务器升级可以在不更改客户端的情况下改进服务
- ✗ No shared data standard — may cause integration issues. 没有共享数据标准——可能导致集成问题
- ✗ Servers may perform repetitive tasks for many clients, reducing efficiency. 服务器可能会为许多客户端执行重复性任务, 从而降低效率

Thin and Fat Clients

- Thin Client: Server does all processing and data work; client handles only display.
服务器负责所有处理和数据处理工作; 客户端仅处理显示
 - Client handles display only.
 - All processing and data work done by the server.
 - Useful when upgrading old systems or reducing client workload. 升级旧系统或减少客户端负载
- ✓ Easy to update centrally. 易于集中更新
- ✓ Less resource use on client side 客户端资源占用更少
- ✗ Heavy load on server. 服务器负载更重
- ✗ Slower response due to network dependency. 由于网络依赖, 响应速度较慢

- Fat (Thick) Client: Server handles data; client handles logic and interface.
服务器处理数据;客户端处理逻辑和界面
 - Client handles UI and logic.
 - Server focuses on data access.
 - Better for rich user experience and offline processing.更有利于丰富的用户体验和离线处理
 - ✓ Fast local processing. 快速的本地处理
 - ✓ Better performance for interactive apps. 更适合交互式应用
 - ✗ Hard to manage updates (must update all clients). 更新管理困难
 - ✗ Higher demand on client devices. 对客户端设备的要求更高

Advantages of Client-Server

- ✓ Easy to manage data across systems. 易于跨系统管理数据
- ✓ Makes good use of networked computers. 充分利用联网计算机
- ✓ New servers can be added or upgraded easily. 可以轻松添加/升级新服务器

Disadvantages of Client-Server

- ✗ No shared data format, so subsystems may not work well together. 没有共享数据格式, 因此子系统可能无法很好地协同工作
- ✗ Servers might do the same tasks again and again. 可能会重复执行相同的任务
- ✗ No central system to list all services, making discovery hard.
没有中央系统列出所有服务, 导致服务发现困难

Quality Properties of Client-Server

- Interoperability: Different clients and servers can work together. 不同的客户端和服务端可以协同工作
- Modifiability: Server updates can quickly be shared with clients. 服务器更新可以快速与客户端共享
- Availability: Using backup servers can improve access. 使用备份服务器可以提升访问效率
- Reusability: Server services can be used in many apps. 服务器服务可用于多种应用

N-Tier Architecture

- Extends client-server by separating responsibilities into **tiers**: 通过将职责划分为多个层级来扩展
 - Presentation Tier – handles UI
 - Logic Tier – handles application rules
 - Data Tier – handles database and storage
- Tiers usually **run on different servers** and communicate through APIs or messages.
- Each tier **handles a specific job**.
- Tiers are **independent** except for the ones directly above or below. 除其直接上级或下级外, 其他层级都是独立的 更好的扩展性, 以异步方式相互通信
- Tiers usually talk to each other asynchronously for better scaling.

*Use when: Building **scalable enterprise applications** that need clear separation and team collaboration.* 构建需要明确职责划分和团队协作的可扩展企业级应用程序

- ✓ Easy to maintain and scale each tier separately. 易于维护和单独扩展
- ✓ Improves flexibility and supports large systems.
- ✗ Increases complexity and communication overhead.
- ✗ Requires good design to avoid bottlenecks or tight coupling. 需要良好的设计

N-Tier & N-Layer

- **N-Tier**: Logical and physical separation (runs on different machines/servers). 逻辑和物理分离 (运行在不同的机器/服务器上)
- **N-Layer**: Code structure separated logically, may still run on same machine. 代码结构在逻辑上分离, 但仍可能在同一台机器上运行

When to Use N-Tier

- Use N-tier when one layer needs a lot of resources, which could slow others down. 当某一层需要大量资源, 这可能会降低其他层的速度时
- Use it when a layer handles private data that other layers shouldn't access. 当某一层处理其他层不应访问的私有数据时

3-Tier Architecture

- Each layer can run on its own processor. 每一层都可以在自己的处理器上运行
- Faster than thin-client and easier to manage than fat-client.
- Scalable 可扩展 – extra servers can be added as needed.

Advantages of N-Tier

- ✓ **Maintainability**: Each tier can be updated without affecting others.
- ✓ **Scalability**: Tiers are modular and can be scaled easily. 模块化设计, 易于扩展
- ✓ **Flexibility**: Each tier can be managed or changed on its own. 单独管理或更改
- ✓ **Availability**: You can add more of one tier to handle more load. 添加更多层以处理更多负载

Disadvantages of N-Tier

- **Bloat 臃肿**: Programs can get large due to inter-tier communication. 由于层间通信, 程序可能会变得庞大
- **Inflexibility**: Sometimes data must move in a certain way across tiers, limiting options. 有时数据必须以特定方式跨层移动, 从而限制了选项

Broker Architecture

- Adds a **middle component (broker)** between clients and servers.
负责定位正确的服务器, 发送客户端请求并返回响应
- The broker locates the right server, sends client requests, and returns responses.

*Use when: You need **flexible and scalable communication** in large distributed systems.*

您需要在大型分布式系统中实现灵活且可扩展的通信

- ✓ Supports cross-platform communication. 支持跨平台
- ✓ Allows reuse of services across many clients.
- ✗ Adds one more layer to manage.
- ✗ Broker becomes a critical failure point if not handled well. 会成为关键的故障点

Quality Properties of Broker

- **Interoperability**: Clients and servers on different platforms can work together.
- **Modifiability**: Server changes are easier to manage.
- **Portability**: Porting the broker lets new clients use the system.
- **Reusability**: Broker helps reuse complex services in other apps.

Peer-to-Peer (P2P)

- All nodes (peers) are **equal** — each one can act as both client and server.
所有节点(对等体)都是平等的——每个节点都可以充当客户端和服务端
- Resources are shared directly between peers without a central server.
资源在对等体之间直接共享, 无需中央服务器

*Use when: Building **file-sharing systems**, decentralized apps, or messaging systems.*

构建文件共享系统、去中心化应用或消息传递系统

- Advantages:
 - ✓ No central point of failure — more fault-tolerant.
没有中心故障点——容错能力更强
 - ✓ Cost-effective and scalable for large user bases.
经济高效, 可扩展性强, 适用于大型用户群
- Disadvantages:
 - ✗ Hard to control or manage. 难以控制或管理
 - ✗ Security and backup are more difficult. 安全性和备份难度更大

Service-Oriented Architecture (SOA)

- System is built from **independent services** that communicate via messages.
系统由通过消息通信的独立服务构建而成
- Each service is **self-contained**, reusable, and can run independently.
系统由通过消息通信的独立服务构建而成
- Commonly implemented using **Web Services (SOAP, REST)**.

*Use when: You need **flexibility, integration with other systems**, or reuse across multiple platforms.* 您需要灵活性、与其他系统集成或跨平台重用

- ✓ **Easy to integrate with different technologies.** 易于与不同技术集成
- ✓ **Services are loosely coupled and easier to maintain or replace.**
- ✗ Harder to design and manage without strong planning. 没有周密的规划
- ✗ Security must be handled at the service level, not globally. 安全性必须在服务级别处理, 而不是全局处理

What is a Service in SOA

- A service is a **complete unit of functionality**, such as "payment", "authentication", or "email sender". 服务是一个完整的功能单元
- Services **talk to each other** using messages (usually over HTTP or queues).
- Can be located **anywhere** — internal or external to the organization.

SOA and Web Services

- Web Services are the main technology for connecting services in SOA.
- Services are linked using Web Services tools. 进行链接
- Services, both inside and outside a company, form the SOA system. 内部和外部的服务构成了 SOA 系统

Basic SOA

- A service consumer sends a message request to a service provider. 服务使用者向服务提供者发送消息请求
- The provider replies with a message. 提供者回复消息
- Both understand the format of the message exchange. 双方都理解消息交换的格式

Advantages of SOA

- **Flexibility:** Loosely connected services are easier to integrate.
- **Interoperability:** Services can work together even if built with different tools.
- **Reusability & Maintainability:** Changes to one service don't affect others, and services can be reused in new systems.

Disadvantages of SOA

- Not good for user interfaces like maps. 不适合地图等用户界面
- Not useful for simple, short apps like word processors.
- Harder to understand and manage.
- Security must be handled at the service level, not app level.

5. Interactive Systems 交互系统

- Designed for **user interaction**, focusing on input, display, and user experience.
专为用户交互而设计, 注重输入、显示和用户体验
- These systems often involve **frequent UI updates** and **real-time feedback**.
这些系统通常需要频繁的 UI 更新和实时反馈
- Examples: Games, simulations, mobile apps, web applications.

*Use when: The system needs to support **user-driven behavior** and quick response to inputs.*

- ✓ Supports high usability and rich user interfaces. 高可用性和丰富的UI
- ✓ Flexible for building modern apps with good UI/UX.
- ✗ Can be complex to manage if logic is mixed with the interface.
- ✗ Requires careful separation to keep code maintainable. 需要仔细分离以确保代码可维护

Interactive System Structure 交互系统结构

- The goal is to separate user interaction from data and logic. 将用户交互与数据和逻辑分离
- Divided into three main functional modules:
 - Data Module – Manages application data and rules. 管理应用程序数据和规则
 - Control Module – Controls system flow, handles events and user decisions.
控制系统流程, 处理事件和用户决策
 - View Module – Manages display, visuals, and collects user input.
管理显示、视觉效果并收集用户输入

Model-View-Controller (MVC)

- Popular pattern used in interactive systems to **manage complex UIs**. 管理复杂 UI
- Splits system into three independent parts to **increase clarity and reusability**. 以提高清晰度和可重用性
- Divides the system into three parts:
 - Model: Core logic and data
 - View: Output to the user (graphical or text)
 - Controller: Handles user input and app flow

MVC – Model

- The model manages data, logic, and rules. 模型管理数据、逻辑和规则
- It has:
 - Data components with app data
 - Logic for user interfaces
- It works without a specific interface and represents the real-world problem. 它无需特定的界面即可工作, 并代表现实世界的问题
- When data changes, it notifies the view to update and controller to adjust commands.⁴
当数据发生变化时, 它会通知视图进行更新, 并通知控制器调整命令

Example: In an e-commerce site, the Model handles product data, inventory, and prices.

MVC – View

- The view shows data visually (charts, diagrams, etc.). 以可视化的方式显示数据
- It includes display components.
- Gets updates from the Model when data changes and shows it to the user. 当数据发生变化时, 从模型获取更新并将其显示给用户
- Can have **multiple views** of the same data (e.g., table view for admin, card view for customers).

Example: Webpage or app screen showing a product catalog.

MVC – Controller

- Handles user input and converts it into actions for the Model or View.
处理用户输入并将其转换为模型或视图的操作
- Acts as a **bridge between user interaction and system behavior**.
充当用户交互和系统行为之间的桥梁

Example: When a user clicks "Add to Cart", the controller updates the model and refreshes the view.

Advantages of MVC

- ✓ **Modifiability:** You can change the UI (View) without touching the logic (Model).
修改视图而不需要修改模型
- ✓ **Reusability:** Same Model can be used with different Views and Controllers.
同一模型可以用于不同的视图和控制器
- ✓ **Separation of Concerns:** Clear boundaries between data, display, and user input handling.
清晰地划分数据、显示和用户输入处理之间的界限
- ✓ **Scalability:** Supports large and complex UI systems by dividing responsibilities.
通过划分职责, 支持大型复杂的 UI 系统

Disadvantages of MVC

- **High Complexity:** Can be overkill for simple applications. 对于简单的应用程序来说可能过于复杂
- **Steep Learning Curve:** Beginners may find it difficult to connect all three parts correctly.
初学者可能难以正确连接所有三个部分
- **Too Many Files:** Requires managing multiple components even for small tasks.
即使是小型任务也需要管理多个组件

Quality Properties of MVC

- **Modifiability:** Changes in one part don't affect others.
- **Usability:** Supports multiple UI views for different users or devices.
- **Maintainability:** Easy to locate and fix bugs since logic, UI, and control are separated.
易于查找和修复错误
- **Testability:** Each part can be tested independently. 可以独立测试